

Aula Revisão:

- **Orientação à Objetos**
- **Recursividade**

Curso: Sistemas de Informação

Disciplina: Algoritmos e Estruturas de Dados

Professor: Fábio Leandro Rodrigues Cordeiro



- **ORIENTAÇÃO À OBJETO**
- **Abstração**
- **Encapsulamento**
- **Objetos**
- **Classes**
- **Classes Publica e Privada**
- **Construtores**
- **Construtor Default**
- **RECURSIVIDADE**
- **Exercícios**

Abstração

Processo de representação de um grupo de entidades através de suas características (atributos) em comuns.

A partir da abstração, cada entidade particular (instância) do grupo é considerada somente pelos seus atributos específicos.

É um agrupamento de subprogramas + dados que é compilado separadamente, de forma independente.

Portanto, é um sistema abstraído.

Muitas vezes os encapsulamentos são colocados em bibliotecas.

O objetivo principal desta técnica é tornar o programa o mais flexível possível, de fácil manutenção.

Abstração x Encapsulamento

A abstração define as entidade que representa um objeto

O encapsulamento esconde detalhes daquele objeto, ocultando detalhes de seu funcionamento



Um Tipo Abstrato de Dados (TAD) é um encapsulamento onde são incluídos somente um tipo específico de dados, além de subprogramas que fornecem as operações para aquele tipo.

Detalhes de implementação do tipo ficam ocultos das unidades fora do encapsulamento que o contém



Objetos

Um objeto é uma variável (instância) de um tipo abstrato de dados, declarada por alguma unidade

Programação orientada a objetos consiste no uso de objetos no desenvolvimento do software

Uma Classe é um tipo de dados semelhante a uma estrutura (struct), constituídas por Atributos e Métodos.

Atributos de classe são semelhantes a campos de estruturas;

Exemplo: uma classe Data poderia conter os atributos *dia*, *mes* e *ano*.

Métodos de classe são funções que operam sobre os atributos.



Classes

Se declararmos uma variável cujo tipo é uma classe, esta variável será composta, assim como nas estruturas.

Haverá uma cópia de cada um dos atributos para cada variável declarada daquela classe.



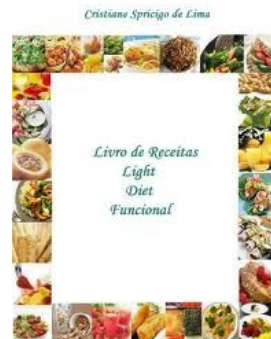
Classes X Objetos

Uma classe é um modelo para algo que deve ser criado.
Por definição, um objeto é a instância de uma classe.

Por exemplo, uma “Receita para Bolo de Chocolate”, será usada como um modelo para o bolo que será criado.

De forma análoga:

*A rece'ita é uma **classe**.*



*O bolo é um **objeto**.*





Classe x Objeto

Exemplo de classe e sua instância (objeto da classe):

```
//Classe Produto  
public class Produto {  
    ...  
}
```

```
//instância da Classe Produto  
Produto meuProduto = new Produto();
```



Uma classe é geralmente dividida em duas partes:

- 1.A parte privada contém elementos que serão de uso exclusivo da classe.
- 2.A parte pública contém os elementos exportados pela classe.

Para simplificar, todos os atributos são privados e todos os métodos são públicos.



Construtores

Uma classe pode possuir também métodos especiais denominados construtores.

Um construtor é um método especial que possui o mesmo nome da classe e não possui tipo de retorno, tendo por finalidade iniciar os atributos do objeto.



```
class Data {  
    private: int dia, mes, ano;  
  
    public: Data(int d, int m, int a) {  
        dia = d; mes = m; ano = a;  
    }  
  
    bool bissexto()  
    {  
        if (ano % 4 != 0) return false;  
        else if (ano % 100 == 0 && ano % 400 != 0) return false;  
        else return true;  
    }  
  
    void imprime()  
    {  
        Console.WriteLine(dia + "/" + mes + "/" + ano);  
    }  
};
```



```
void main()  
{  
    Data d(10,3,2003);  
  
    d.imprime();  
}
```

```
void main()  
{  
    Data d1 = new Data (10,3,2003);  
    d1.imprime();  
}
```



Construtor Padrão (Default)

É um construtor sem parâmetros.

É um método sem parâmetros e sem tipo de retorno e cujo nome é o mesmo da classe.

Este construtor especial serve para iniciar objetos com valores padrão.



```
class Data {  
  
    private: int dia, mes, ano;  
  
    public: Data(int d, int m, int a) {  
        dia = d; mes = m; ano = a;  
    }  
  
    Data() { // construtor default  
        dia = 1; mes = 1; ano = 2003;  
    }  
  
    bool bissexto() {  
        if (ano % 4 != 0) return false;  
        else if (ano % 100 == 0 && ano % 400 != 0) return false;  
        else return true;  
    }  
  
    void imprime() {  
        Console.WriteLine(dia + "/" + mes + "/" + ano);  
    }  
}
```



RECURSIVIDADE

Um objeto é dito recursivo (ou representa recursividade) se for definido em termos de si próprio.

Quando um objeto faz uma chamada a si mesmo é dito
que temos
Uma chamada recursiva





Recursão

É o processo de resolução de um problema, reduzindo-o em um ou mais subproblemas com as seguintes características:

São idênticos aos problemas originais

São mais simples de resolver (componentes)

A solução completa do problema original é obtida através da “montagem” dos componentes



Estrutura da Recursão

Usualmente, uma função recursiva é dividida em:

- Chamada Recursiva
- Condição de Parada

A chamada recursiva pode ser direta (mais comum) ou indireta (A chama B que chama A novamente)

A condição de parada é fundamental para evitar a execução de loops infinitos

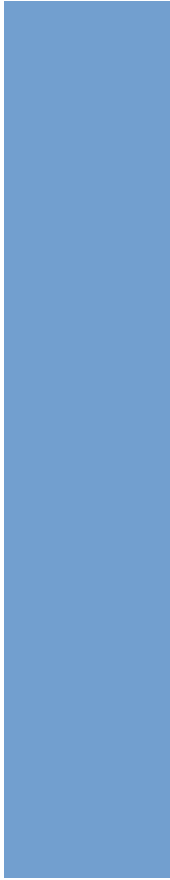


Recursividade

Exemplo: soma dos números (solução 1)

Para $m = 4$ e $n = 7$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n-1) + n );  
}
```



▮ Pilha



Recursividade

Para $m = 4$ e $n = 7$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n-1) + n );  
}
```



▮ Pilha



Recursividade

Para $m = 4$ e $n = 6$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

$m = 4$
 $n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 6$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

$m = 4$
 $n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 5$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

$m = 4$

$n = 6$

$m = 4$

$n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 5$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

$m = 4$

$n = 6$

$m = 4$

$n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 4$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

$m = 4$
 $n = 5$

$m = 4$
 $n = 6$

$m = 4$
 $n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 4$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

$m = 4$
 $n = 5$

$m = 4$
 $n = 6$

$m = 4$
 $n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 4$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

Retorno: 4

$m = 4$
 $n = 5$

$m = 4$
 $n = 6$

$m = 4$
 $n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 5$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

Retorno: $4 + 5 = 9$

$m = 4$

$n = 6$

$m = 4$

$n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 6$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

Retorno: $9 + 6 = 15$

$m = 4$
 $n = 7$

Pilha



Recursividade

Para $m = 4$ e $n = 7$

```
int Soma ( int m, int n ){  
    if ( n == m)  
        return (m);  
    else  
        return ( Soma (m, n - 1) + n );  
}
```

Retorno: $15 + 7 = 22$

Pilha



Chamada de Função

Uma função pode ser chamada por outras funções ou pelo programa principal (função *main*).

A indicação de onde a função foi chamada deve ser armazenada pelo sistema.

Nas chamadas de funções recursivas uma pilha é utilizada para armazenar o estado da cada função, quando esta foi chamada.



Chamada de Função

O estado da função é caracterizado pelo conteúdo de todas as variáveis, seus parâmetros e endereço de retorno.

O endereço de retorno serve para que o sistema “lembre” onde deve retornar a execução depois que a função terminar.



Chamada de Função

Sempre que uma função é iniciada recursivamente, um novo conjunto de variáveis locais e de parâmetros é alocado.

Quando ocorre um retorno, este conjunto é desativa, dando lugar ao conjunto de valores da função que fez a chamada.



Exemplo de função recursiva:

- Função que define elevar qualquer número x a uma potência inteira não negativa n .
- Notação:

$$pot(x, n) = \begin{cases} 1 & \text{se } n = 0 \\ x \times pot(x, n-1) & \text{se } n > 0 \end{cases}$$



Recursividade

Função que calcula x^n pode ser escrita diretamente a partir da definição de uma potência:

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

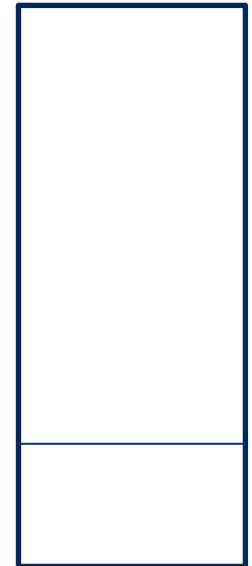


Recursividade

Para $n = 3$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



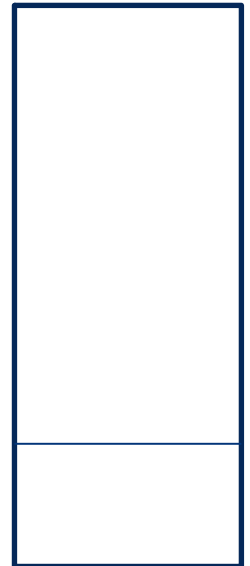


Recursividade

Para $n = 3$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



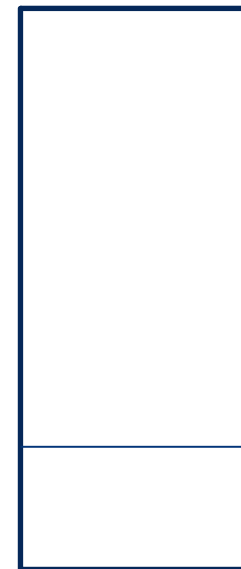


Recursividade

Para $n = 3$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



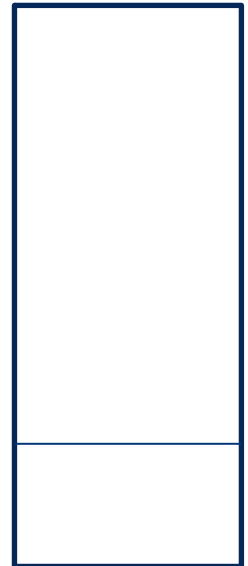


Recursividade

Para $n = 3$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



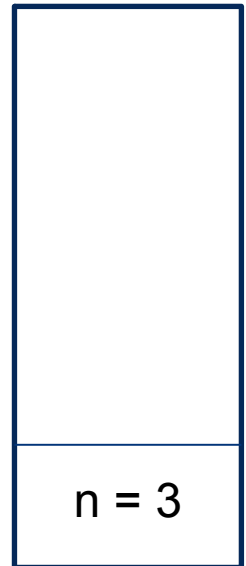


Recursividade

Para $n = 2$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



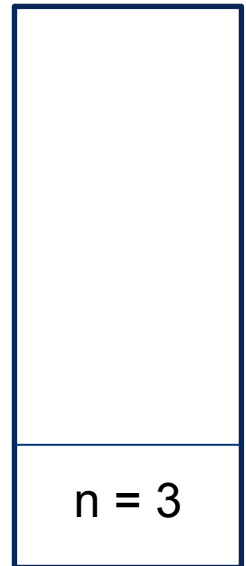


Recursividade

Para $n = 2$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



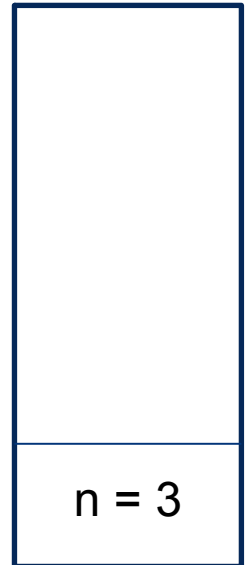


Recursividade

Para $n = 2$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



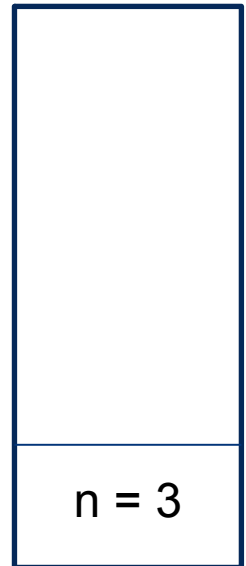


Recursividade

Para $n = 2$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



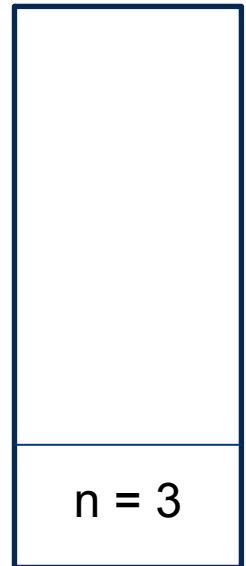


Recursividade

Para $n = 2$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



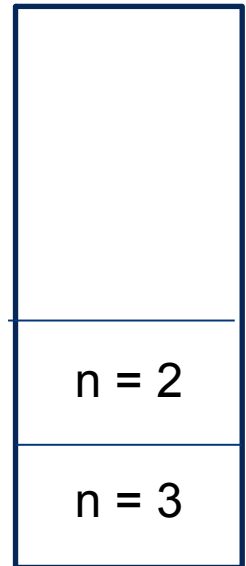


Recursividade

Para $n = 1$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



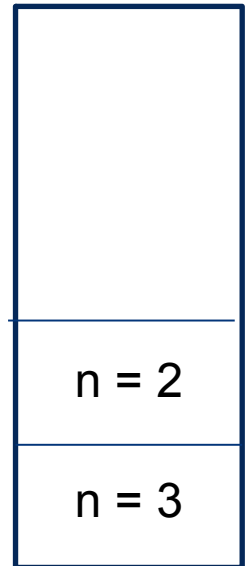


Recursividade

Para $n = 1$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



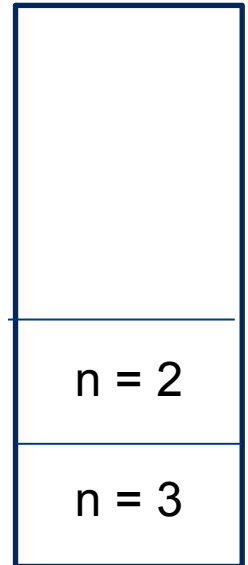


Recursividade

Para $n = 1$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



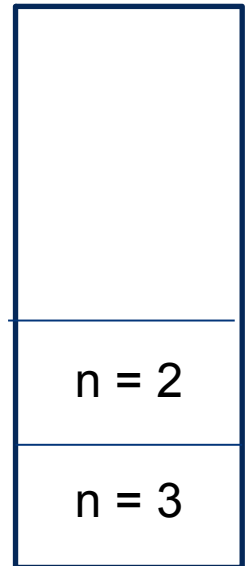


Recursividade

Para $n = 1$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



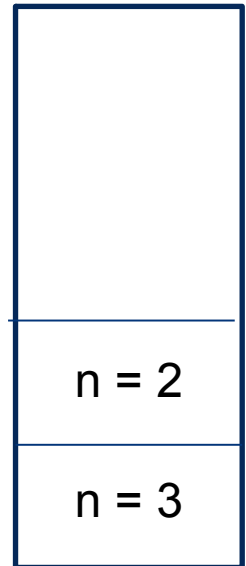


Recursividade

Para $n = 1$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```





Recursividade

Para $n = 0$ e $x = 2$;

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

Pilha

$n = 1$
$n = 2$
$n = 3$

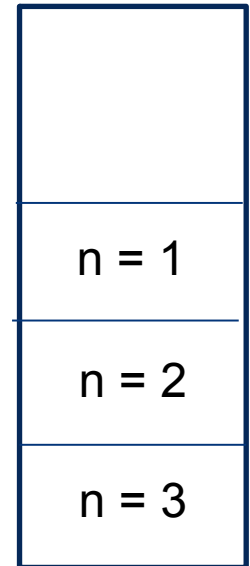


Recursividade

Para $n = 0$ e $x = 2$;

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

Pilha



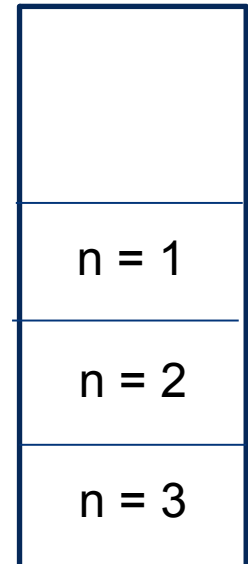


Recursividade

Para $n = 0$ e $x = 2$;

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

Pilha



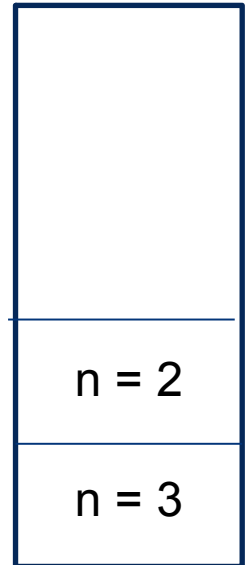


Recursividade

Para $n = 1$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```





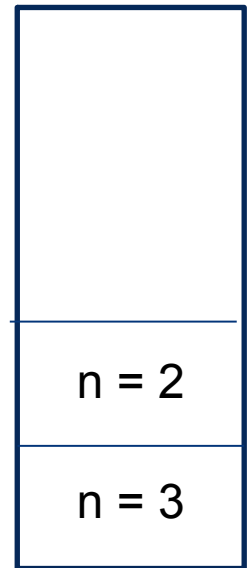
Recursividade

Para $n = 1$ e $x = 2$;

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

$$2 \times 1 = 2$$

Pilha



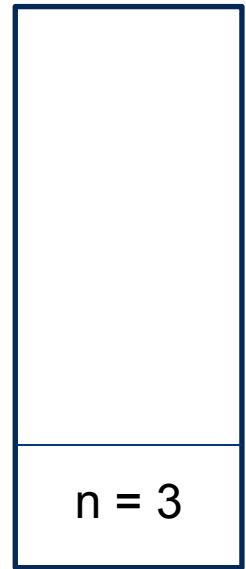


Recursividade

Para $n = 2$ e $x = 2$;

Pilha

```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

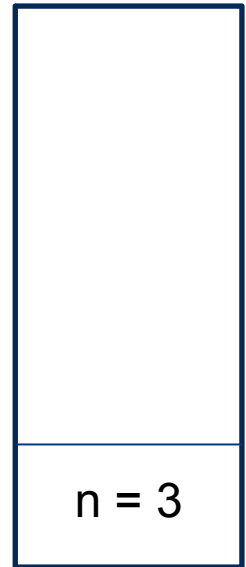




Recursividade

Para $n = 2$ e $x = 2$;

Pilha



```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

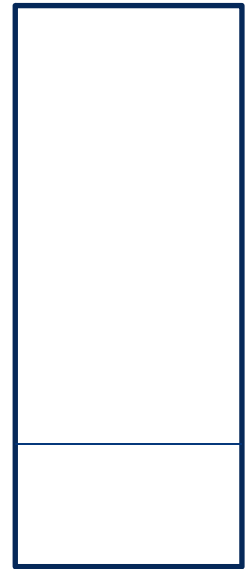
$$2 \times 2 = 4$$



Recursividade

Para $n = 3$ e $x = 2$;

Pilha



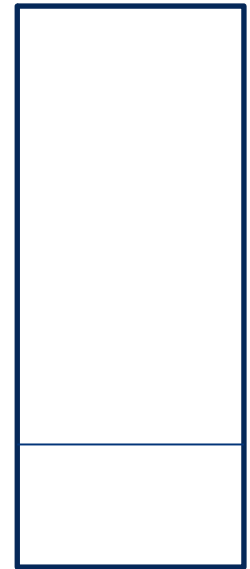
```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```



Recursividade

Para $n = 3$ e $x = 2$;

Pilha



```
double potencia(double x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

$$2 \times 4 = 8$$



- TOSTES, Ana. **Notas de Aula**. Belo Horizonte: PUC Minas, 2013.
- ZIVIANI, Nivio. **Projeto de algoritmos**: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p. ISBN 8522105251 (Disponível no Acervo).



Exercícios

1. Escreva uma classe *Data* contendo as seguintes operações:

- *construtor*: recebe os valores de dia, mês e ano como parâmetros e inicia a data;
- *getters*: métodos para retornar os valores de dia, mês e ano;
- *setters*: métodos para alterar os valores de dia, mês e ano;
- *extenso*: retorna a data por extenso como uma string;
- *proximoDia*: altera a data internamente de modo que esta passará a representar o dia seguinte de uma data (atenção às mudanças de mês e ano).

2. Escreva um programa que leia duas datas e calcule o número de dias de uma data para outra por meio de chamadas sucessivas do método *proximoDia*. No final, você deve imprimir: